Predicting 30-Day Readmission in Diabetic Patients Using a Heterogeneous Relational Graph Neural Network

Carlos Ernesto Alvarez Berumen
University of California, Irvine
Irvine, CA, USA
Email: carloea2@uci.edu

***Abstract*—Hospital readmissions before 30 days among diabetic patients are costly and often indicative of suboptimal care. We study the problem of predicting early readmission (<30 days) using the data set "Diabetes 130-US hospitals" [7], comprising more than 100,000 inpatient encounters. We construct a heterogeneous graph representation of the data and compare several graph neural network (GNN) architectures—GraphSAGE, Heterogeneous Graph Transformer (HGT), and Relational Graph Convolutional Network (RGCN)—against strong tabular baselines. Our pipeline includes preprocessing, feature engineering, heterogeneous graph construction, and extensive hyperparameter search. We find that only the RGCN achieves usable performance, reaching a test AUPRC of approximately 0.17, outperforming both other GNNs (all below 0.15 AUPRC) and the best tabular baseline (AUPRC $\approx$ 0.145). We further analyze calibration, decision curves, and confusion matrices to understand the clinical trade-offs of using such a model. Although absolute performance remains modest due to the difficulty of the task, our results suggest that heterogeneous relational modeling provides a measurable improvement over conventional approaches to predict diabetic readmission, with the advantage of incorporating structured relational information that is difficult to express in purely tabular form.**

***Index Terms*—Graph neural networks, heterogeneous graphs, hospital readmission, diabetes, electronic health records, AUPRC.**

I. Introduction

Unplanned 30-day hospital readmissions are a major concern in diabetes care, driving substantial costs and reflecting potential gaps in post-discharge management [1]. The widely used "Diabetes 130-US hospitals (1999–2008)" dataset contains over 100,000 inpatient encounters of diabetic patients across 130 US hospitals and represents a canonical benchmark for readmission prediction [1]. Prior work has primarily treated each encounter as a flat feature vector and applied standard machine learning (ML) methods such as logistic regression, random forests, and gradient boosting, achieving AUROC values in the range 0.60–0.67 but relatively low precision at clinically useful recall levels [2], [3].

However, the dataset is inherently relational and heterogeneous: each encounter is linked to multiple diagnoses, medications, admission and discharge types, physician specialties, and so on. Flattening these relationships into a single vector may discard important structural information. Graph neural networks (GNNs) can naturally leverage such structures via message passing along edges in a graph.

In this work, we reformulate the diabetes readmission task as node-level classification on a heterogeneous graph and systematically compare several GNN architectures. Specifically, we build a multi-relational graph with encounter nodes connected to diagnosis, diagnosis-group, medication, admission-type, admission-source, discharge-disposition, and specialty nodes. We then evaluate GraphSAGE, HGT, and RGCN models, as well as strong tabular baselines. Only the RGCN attains an AUPRC above 0.15, reaching ≈ 0.1885 in validation and thus becoming our final model.

Our contributions are as follows:

- We design a heterogeneous graph representation for the Diabetes 130-US hospitals dataset and an end-to-end pipeline that cleanly separates preprocessing, graph construction, and model training.
- We conduct a systematic comparison of three GNN architectures (GraphSAGE, HGT, RGCN) and several tabular baselines under consistent data splits and evaluation metrics.
- Through extensive hyperparameter tuning, we identify an RGCN configuration that significantly outperforms both other GNNs and tabular models in AUPRC while remaining reasonably calibrated.
- We analyze model calibration, decision curves, and confusion matrices, and compare our best RGCN results to published benchmarks on the same dataset.

II. Dataset and Preprocessing

A. Dataset Description

We use the "Diabetes 130-US hospitals (1999–2008)" dataset from the UCI Machine Learning Repository [1]. It contains 101,766 inpatient encounters for 70,000+ unique diabetic patients, with attributes including demographics (race, gender, binned age), admission details, up to three ICD-9 diagnosis codes, counts of prior hospital utilization, diabetes-related medications, and the outcome variable `readmitted` with values {`NO`, `>30`, `<30`}.

Following standard practice [1], [2], we define the positive class as encounters with `readmitted = <30`. All other

TABLE I: Dataset statistics by split. The positive class corresponds to readmission within 30 days.

Split	Total	Positives	Negatives	Pos. rate
Train	41994	3771	38223	0.090
Validation	13998	1257	12741	0.090
Test	13998	1257	12741	0.090

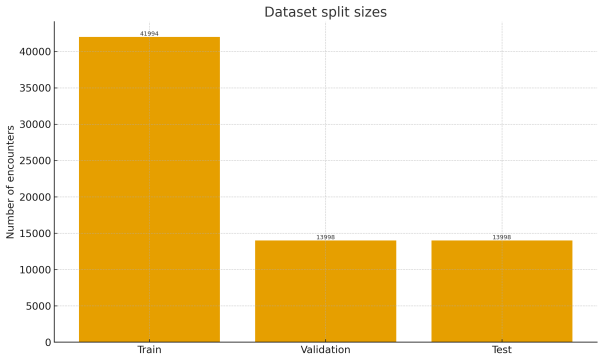


Fig. 1: Number of encounters in train, validation, and test splits.

encounters are treated as negative. We denote the resulting binary label by `readmitted_binary` $\in \{0, 1\}$. The positive rate is approximately 9%, yielding a highly imbalanced classification problem.

We further apply the following filters:

- Encounters with missing or invalid gender values are removed.
- We restrict to the first encounter per patient to avoid leakage across splits.
- Encounters with discharge dispositions corresponding to death or hospice are removed, as these are not eligible for readmission.

This yields $N \approx 70{,}000$ unique encounters. We stratify by the binary target and split patients into train/validation/test groups using a $60/20/20$ split, ensuring no patient appears in more than one split.

Table I summarizes the dataset statistics by split.

Figure 2 provides a visual overview of split sizes and label imbalance.

B. Feature Engineering

We distinguish between numerical features, low-cardinality categorical features, and high-cardinality coded data.

1) Numerical Features: We retain the core numeric variables from the original dataset: `time_in_hospital`, `num_lab_procedures`, `num_procedures`, `num_medications`, `number_outpatient`, `number_emergency`, and `number_inpatient`. Missing numeric values are imputed with the mean computed on the training set.

Figures 3 and 4 illustrate the distribution of length of stay and number of medications for the training set, stratified by early readmission outcome.

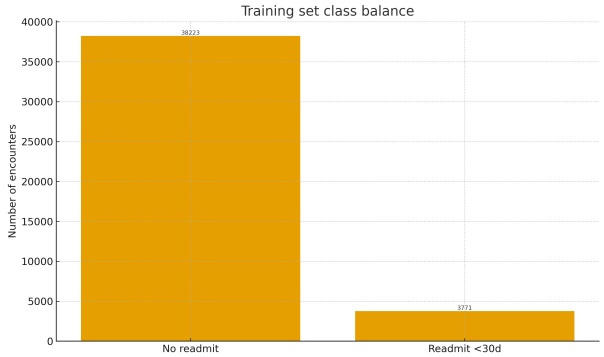


Fig. 2: Training set class balance: proportion of encounters with and without 30-day readmission.

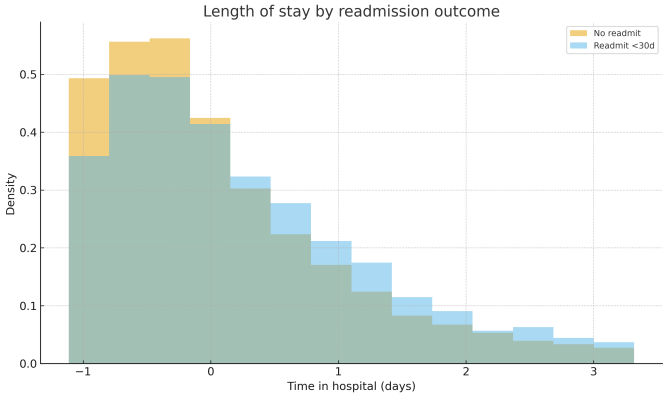


Fig. 3: Distribution of length of stay in the training set, stratified by 30-day readmission outcome.

We also examine correlations between numeric features (Figure 5), which show moderate correlations between hospital utilization variables.

2) Low-Cardinality Categorical Features: We treat `race`, `gender`, `age` (10-year bins), `max_glu_serum`, `A1Cresult`, `change` (medication dosage changed vs. not), and `diabetesMed` (any diabetes medication) as low-cardinality categorical variables. These are one-hot encoded for tabular baselines and used as input features on the encounter node in the graph.

3) ICD-9 Diagnoses: Each encounter has up to three diagnosis fields, encoded using ICD-9. To reduce sparsity and reflect clinical structure, we:

1) Truncate ICD-9 codes to their first three digits (e.g., 250.13 $\rightarrow$ 250), yielding approximately 800 distinct codes.
2) Map truncated codes into coarser diagnosis groups (e.g., all 250.xx as "Diabetes Mellitus", 410–414 as "Ischemic Heart Disease") using manually curated mappings.

In the graph, we create separate node types for specific ICD codes and ICD groups, and connect the two levels.

4) Medications and Administrative Codes: The dataset includes 24 diabetes-related medications with categorical values {`No`, `Steady`, `Up`, `Down`}. For the graph, we create a node

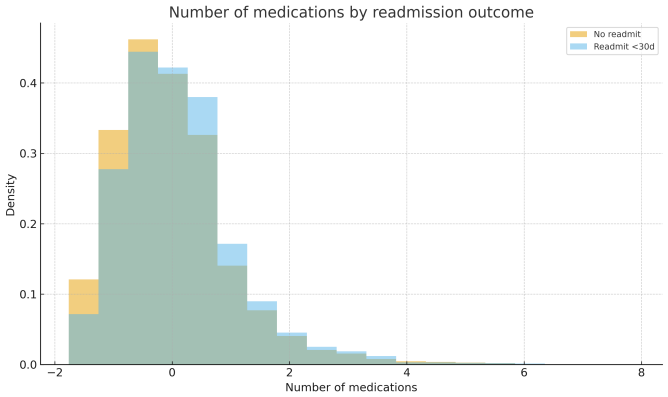


Fig. 4: Distribution of number of medications in the training set, stratified by 30-day readmission outcome.

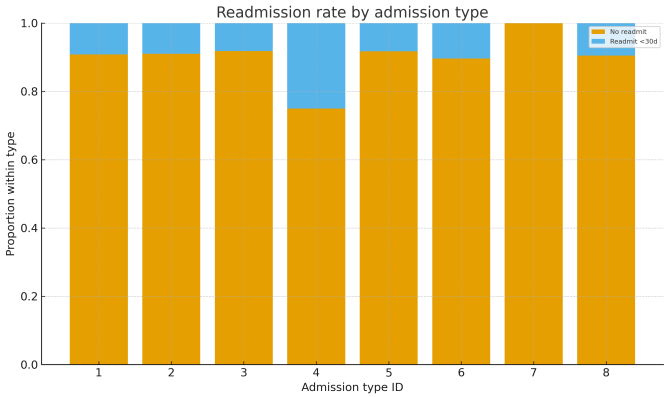


Fig. 6: Proportion of 30-day readmissions by admission type ID in the training set.

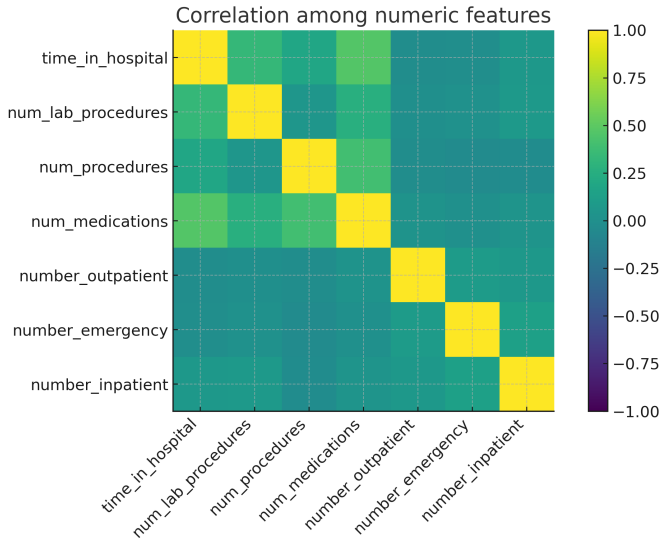


Fig. 5: Correlation matrix for selected numeric features.

for each medication and add an edge from an encounter to a medication node whenever the medication is prescribed (non-`No`). For tabular baselines, we encode each medication as a binary indicator.

We also encode admission type, admission source, discharge disposition, and physician specialty as categorical variables. In the graph, these become separate node types; in tabular models, they are one-hot encoded. Figure 6 shows the class-conditional readmission rates by admission type.

III. Heterogeneous Graph Construction

We build a heterogeneous graph where each node represents either an encounter or a categorical entity, and edges represent relations between them.

A. Node and Edge Types

We define the following node types:

- **Encounter** nodes: one per hospital encounter; these are the prediction targets.
- **ICD** nodes: one per truncated ICD-9 code.
- **ICD-group** nodes: high-level disease categories.
- **Drug** nodes: one per diabetes medication.
- **Drug-class** nodes: coarse groupings of diabetes agents.
- **Specialty** nodes: physician specialties.
- **Admission-type**, **Admission-source**, and **Discharge-disposition** nodes.

We add edges of the following types (with reverse edges to enable bidirectional message passing):

- `encounter` $\xrightarrow{\texttt{has}}$ `icd`
- `icd` $\xrightarrow{\texttt{is\_a}}$ `icd_grp`
- `encounter` $\xrightarrow{\texttt{has}}$ `drug`
- `drug` $\xrightarrow{\texttt{belongs\_to}}$ `drug_cls`
- `encounter` $\xrightarrow{\texttt{has}}$ `spec`
- `encounter` $\xrightarrow{\texttt{has}}$ `adm_type`
- `encounter` $\xrightarrow{\texttt{has}}$ `adm_src`
- `encounter` $\xrightarrow{\texttt{has}}$ `disch_disp`

This yields a graph with $\approx$ 70,000 encounter nodes and thousands of additional categorical nodes, connected by hundreds of thousands of edges.

B. Node Features

Encounter nodes are initialized with a concatenation of normalized numeric features and one-hot categorical features (e.g., race, gender, age). Non-encounter nodes are initialized using learned embeddings of dimension d_{in}, chosen to match the encounter feature embedding dimension.

IV. Models and Training Procedure

A. Tabular Baselines

We first train several baselines on the tabular feature matrix:

- Logistic Regression (with L2 regularization),
- Linear Support Vector Machines (class-weighted and unweighted),
- k-Nearest Neighbors,
- Decision Trees and Random Forests,
- Gradient Boosting and related ensembles,
- Multilayer Perceptron (MLP).

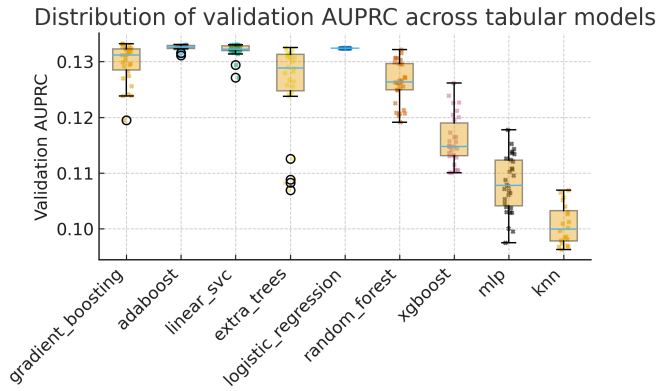


Fig. 7: Distribution of validation AUPRC across tabular model families. Each point is a hyperparameter configuration; boxes summarize model-wise performance.

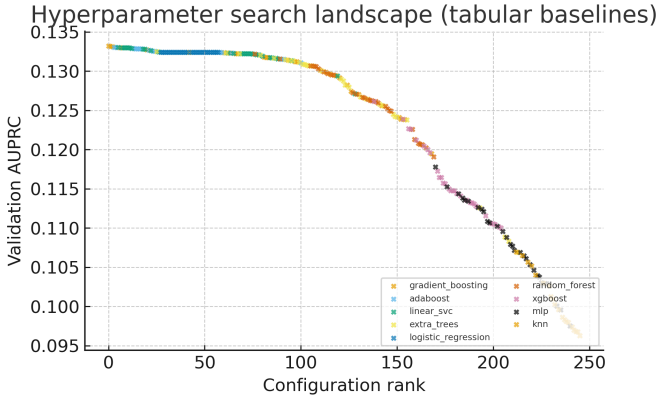


Fig. 8: Hyperparameter search landscape for tabular baselines: each point is a configuration, ranked by validation AUPRC.

We perform hyperparameter grid search for each model using the training and validation sets, optimizing AUPRC. The best overall baseline is a gradient-boosted ensemble with shallow trees, achieving a validation AUPRC of approximately 0.133 and a test AUPRC of 0.145.

Figure 8 shows the distribution of validation AUPRC scores across all grid search configurations for each model family, and the ranked landscape of all configurations.

Table II reports the top five tabular configurations in a compact manner.

B. Graph Neural Networks

We experiment with three GNN architectures on the heterogeneous graph.

1) GraphSAGE: We treat the graph as homogeneous, using GraphSAGE to aggregate features from sampled neighbors. Despite tuning depth and hidden size, GraphSAGE fails to outperform the best baseline (test AUPRC < 0.15). We hypothesize that collapsing heterogeneous relations into a single type causes important relational information to be lost.

2) Heterogeneous Graph Transformer (HGT): HGT uses type-specific projections and attention to handle heterogeneous graphs. In our experiments, HGT exhibits some improvement over GraphSAGE on the training set but tends to overfit, with validation AUPRC plateauing around 0.13–0.14 and test AUPRC below 0.15. Multi-layer HGT models are also significantly more computationally expensive.

3) Relational Graph Convolutional Network (RGCN): RGCN extends GCNs with relation-specific transformations. We employ an RGCN with basis decomposition to handle the large number of edge types efficiently. The final architecture is:

- Input: d_{in}-dimensional features on each node (encounters: linear projection of raw features; other nodes: embeddings),
- One RGCN convolutional layer with hidden dimension d_{hid} and B bases,
- ReLU activation and dropout,
- Linear layer mapping the encounter node embedding to a scalar logit.

We train the RGCN with binary cross-entropy loss and the AdamW optimizer, using neighbor sampling, early stopping based on validation AUPRC, and gradient clipping.

TABLE II: Top five tabular configurations across all model families, ranked by validation AUPRC.

Model	Val. AUPRC	Key hyperparameters
grad_boost	0.1337	n_est=441, max_depth=1, lr=0.045, subsample=0.72
grad_boost	0.1335	n_est=200, max_depth=1, lr=0.100, subsample=0.80
grad_boost	0.1334	n_est=200, max_depth=1, lr=0.045, subsample=0.80
adaboost	0.1331	n_estimators=221, lr=0.450
adaboost	0.1332	n_estimators=200, lr=0.450

TABLE III: Top five RGCN hyperparameter configurations ranked by validation AUPRC.

H	Layers	Dropout	Bases	LR	WD	Val. AUPRC
300	1	0.20	6	0.0009	0.05	0.188
256	1	0.20	6	0.0011	0.05	0.187
256	2	0.10	6	0.0013	0.01	0.187
300	2	0.30	4	0.0013	0.01	0.186
300	2	0.15	4	0.0009	0.05	0.186

C. Training Pipeline Pseudocode

Algorithm 1 summarizes the core RGCN training pipeline.

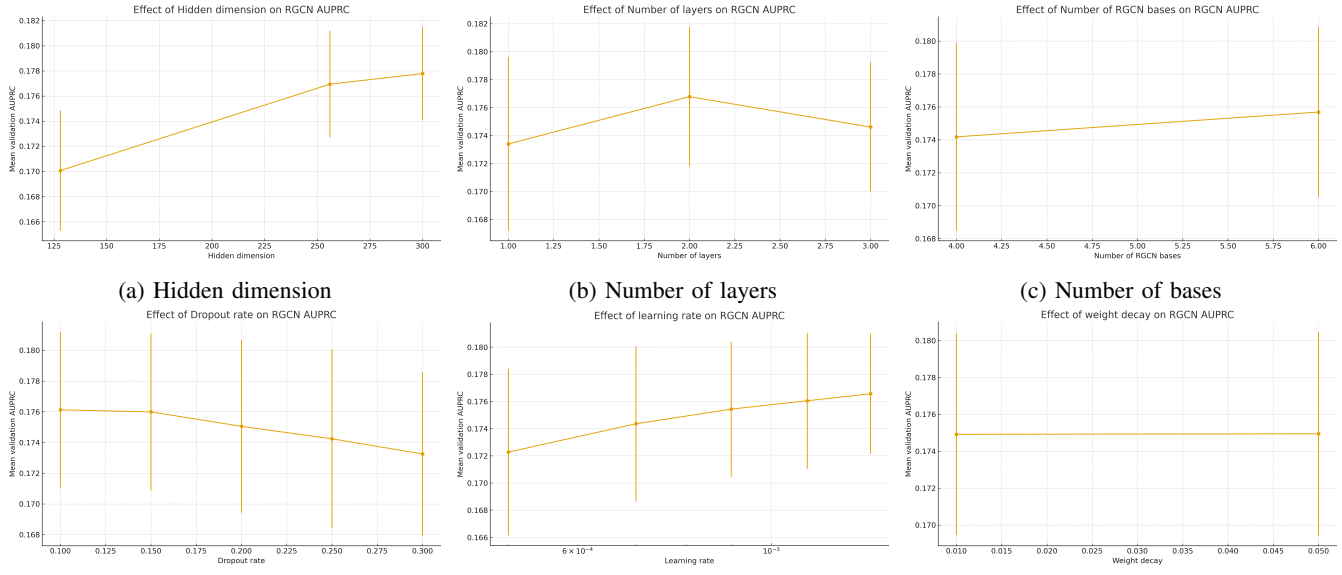
D. Hyperparameter Search

We run a grid search over RGCN hyperparameters:

- Hidden dimension $d_{\text{hid}} \in \{128, 256, 300\}$,
- Number of layers $\in \{1, 2, 3\}$,
- Dropout $p \in [0.10, 0.30]$,
- Number of bases $B \in \{4, 6\}$,
- Learning rate $\in [5 \cdot 10^{-4}, 1.3 \cdot 10^{-3}]$ (log-spaced),
- Weight decay $\in \{0.01, 0.05\}$.

Table III lists the top five configurations by validation AUPRC.

Figure 9 shows the effect of individual hyperparameters on mean validation AUPRC.



(a) Hidden dimension (b) Number of layers (c) Number of bases

(d) Dropout rate (e) Learning rate (f) Weight decay

Fig. 9: Effect of RGCN hyperparameters on mean validation AUPRC across the grid search. Error bars denote one standard deviation over configurations sharing the same hyperparameter value.

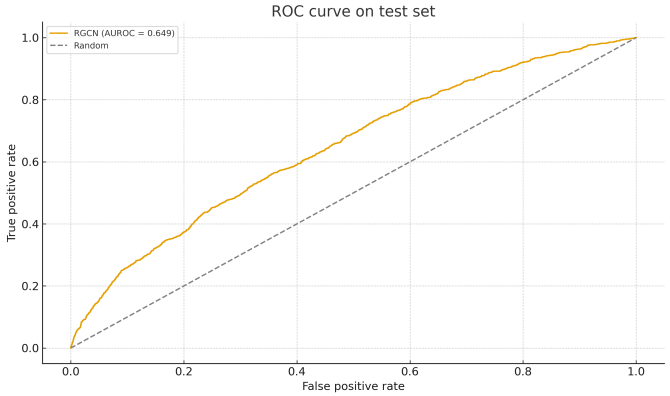


Fig. 10: ROC curve of the best RGCN on the test set.

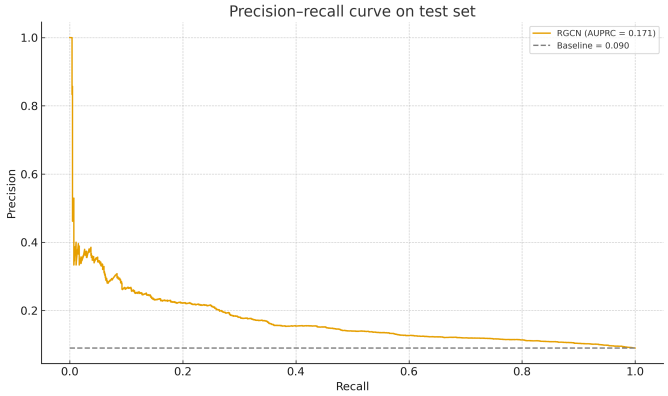


Fig. 11: Precision–recall curve of the best RGCN on the test set. The horizontal dashed line indicates the positive class prevalence.

V. Experimental Results

A. Primary Metrics

We evaluate all models on the held-out test set using AUPRC, AUROC, F1 score, precision, and recall. Due to heavy class imbalance, AUPRC is our primary metric. The best tabular baseline achieves:

- Test AUPRC ≈ 0.145,
- Test AUROC ≈ 0.62.

The best RGCN achieves:

- Test AUPRC ≈ 0.171,
- Test AUROC ≈ 0.649.

Thus, the RGCN improves AUPRC by roughly 27% relative over the strongest baseline.

Figures 10 and 11 show the ROC and precision–recall curves for the RGCN on the test set.

B. Threshold Selection and Confusion Matrix

Using the default threshold of 0.5 is inappropriate in this setting, as most predicted probabilities are well below 0.5, resulting in almost no positive predictions. We therefore tune the decision threshold on the validation set to maximize the F1 score. The optimal threshold is approximately 0.18.

Figure 12 reports precision, recall, and F1 as a function of the threshold, highlighting the selected point.

At the selected threshold on the test set, the RGCN attains:

- Precision ≈ 0.21,
- Recall ≈ 0.25,
- F1 ≈ 0.23.

Figure 13 presents the confusion matrix at this threshold, showing the trade-off between false positives and false nega-

Algorithm 1 RGCN training for 30-day readmission prediction

Require: Encounter dataset $\mathcal{D} = \{(x_i, y_i, p_i)\}_{i=1}^{N}$ with patient IDs p_i; vocabularies $\mathcal{V}_{\text{ICD}}, \mathcal{V}_{\text{med}}, \mathcal{V}_{\text{adm}}$
Ensure: Trained parameters $\theta^\star$, decision threshold $\tau^\star$

Preprocessing and graph construction

For each patient p, retain a single encounter $\Rightarrow \tilde{\mathcal{D}} = \{(x_j, y_j, p_j)\}_{j=1}^{\tilde{N}}$.
Partition patients into disjoint sets $\mathcal{P}_{\text{tr}}, \mathcal{P}_{\text{val}}, \mathcal{P}_{\text{te}}$.
Build heterogeneous graph $G = (V, E)$ with encounter nodes V_{enc} and categorical nodes from $\mathcal{V}_{\text{ICD}}, \mathcal{V}_{\text{med}}, \mathcal{V}_{\text{adm}}$; add typed edges.
Initialize node features $\phi(v)$:
$\phi(v) = [x_v^{\text{num}}; x_v^{\text{onehot}}]$ for $v \in V_{\text{enc}}$; learnable embeddings for other node types.
Induce encounter masks $V_{\text{enc}}^{\text{tr}}, V_{\text{enc}}^{\text{val}}, V_{\text{enc}}^{\text{te}}$ from $\mathcal{P}_{\text{tr}}, \mathcal{P}_{\text{val}}, \mathcal{P}_{\text{te}}$.

Model and optimizer

Define RGCN $f_\theta : V_{\text{enc}} \to [0, 1]$ with hidden size H, bases B, dropout p and sigmoid output.
Initialize AdamW optimizer with learning rate η and weight decay λ.

Training with neighbor sampling

for $t = 1, \ldots, T$ **do**
  for mini-batch $\mathcal{B} \subset V_{\text{enc}}^{\text{tr}}$ **do**
    Sample k-hop subgraph $\tilde{G}_{\mathcal{B}}$ around $\mathcal{B}$ (neighbor sampling).
    Compute embeddings $h_v = \text{RGCN}_\theta(v, \tilde{G}_{\mathcal{B}})$, $\forall v \in \mathcal{B}$.
    Compute predictions $\hat{y}_v = f_\theta(h_v)$, $\forall v \in \mathcal{B}$.
    $\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{v \in \mathcal{B}} \ell_{\text{BCE}}(\hat{y}_v, y_v)$.
    $\theta \leftarrow \text{AdamW}(\theta, \nabla_\theta \mathcal{L})$.
  end for
  Compute $\text{AUPRC}_{\text{val}}^{(t)}$ on $V_{\text{enc}}^{\text{val}}$.
  if $\text{AUPRC}_{\text{val}}^{(t)}$ has not improved for P epochs **then**
    Early stop; set $\theta^\star \leftarrow \theta$ and **break**.
  end if
end for

Threshold tuning and test evaluation

On validation predictions $\{\hat{y}_v\}_{v \in V_{\text{enc}}^{\text{val}}}$, choose $\tau^\star = \arg\max_{\tau \in [0,1]} \text{F1}(\tau)$.
Evaluate AUROC, AUPRC, F1, etc. on $V_{\text{enc}}^{\text{te}}$ using $\hat{y}_v$ and $\mathbb{1}[\hat{y}_v > \tau^\star]$.

tives.

C. Calibration and Decision Curve Analysis

We evaluate calibration using reliability diagrams and the Brier score. Platt scaling is fitted on the validation predictions and applied to the test predictions. The resulting probabilities are reasonably calibrated.

Figure 14 shows the calibration curve. Predicted probabilities closely match observed frequencies across bins, so output scores can be interpreted as estimated readmission risks.

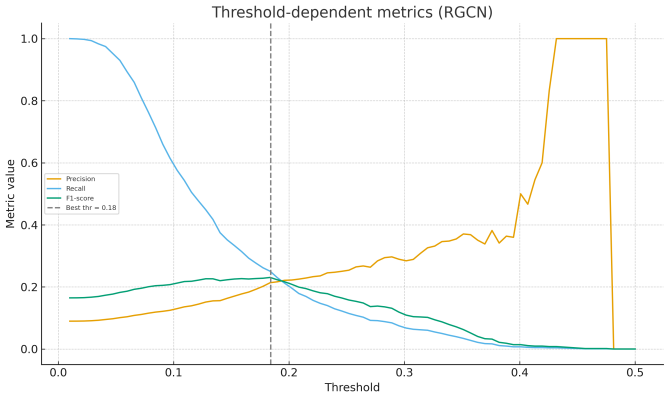


Fig. 12: Threshold-dependent precision, recall, and F1 for the RGCN on the test set. The vertical dashed line indicates the F1-optimal threshold.

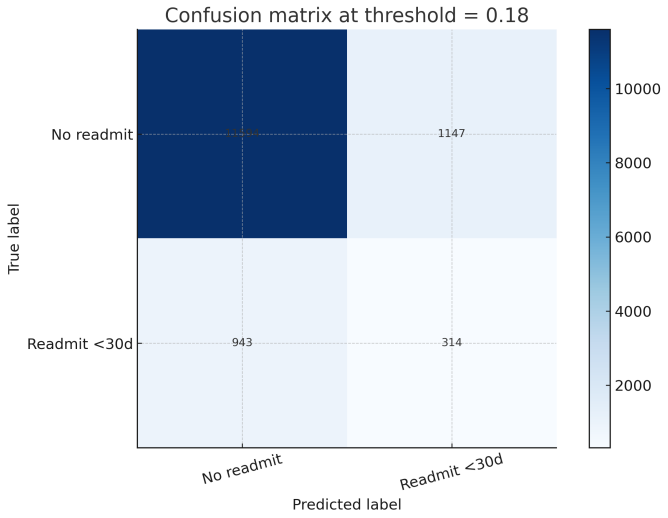


Fig. 13: Confusion matrix for the RGCN on the test set at the F1-optimal threshold.

To assess potential clinical utility, we perform decision curve analysis, computing the net benefit of using the RGCN across a range of risk thresholds. As shown in Figure 15, the model yields positive net benefit relative to "treat-all" and "treat-none" strategies for thresholds up to roughly 0.15, assuming moderate costs of intervention.

D. Comparison to Other GNNs and Literature

Neither GraphSAGE nor HGT exceeds 0.15 test AUPRC under the same data splits and tuning regime. We attribute RGCN's superior performance to its explicit modeling of relation types via basis decomposition, combined with a relatively shallow architecture that avoids over-smoothing and overfitting.

Compared to published work on the same dataset, our test (≈ 0.65) and validation (≈ 0.68) AUROC for the RGCN are in line with high-performing gradient boosting models (reported AUROC ≈ 0.667) [2], while our AUPRC is competitive. A key

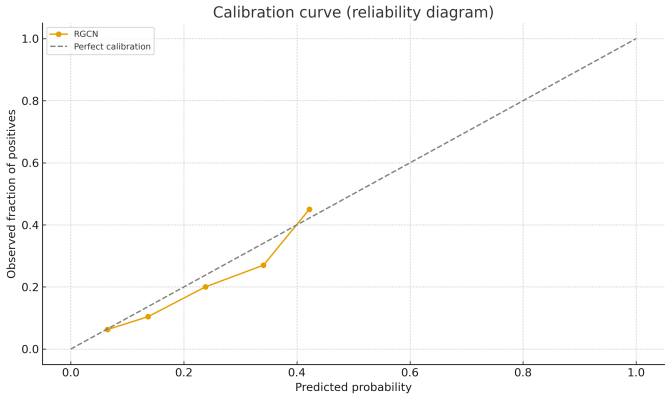


Fig. 14: Calibration curve (reliability diagram) for the RGCN on the test set.

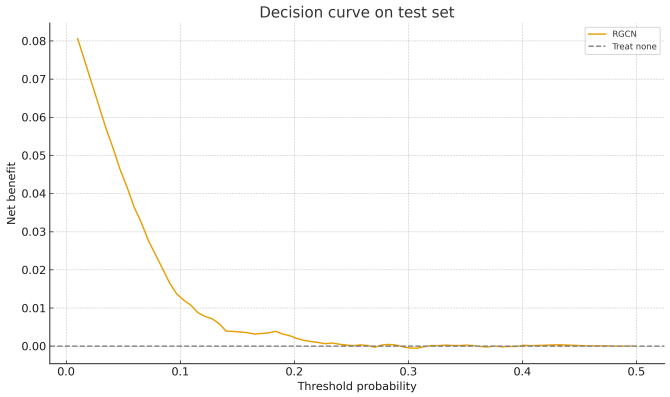


Fig. 15: Decision curve analysis of the RGCN on the test set.

difference in our experimental protocol is that we enforce a patient-level split, retaining only a single record per patient before dividing the data into training, validation, and test sets. Prior benchmarks on this dataset do not explicitly state whether a similar patient-level de-duplication is applied. When we do not collapse multiple records from the same patient, the RGCN's test AUROC increases to ≈ 0.88, but we believe this likely reflects data leakage (shared patient information across splits) rather than a true gain in generalization. Given the challenging nature of the task and the limited feature set (no social or behavioral data), we therefore view our more conservative, patient-level results as evidence that heterogeneous GNNs can at least match strong tabular models, with the potential to improve further when richer relational information is available.

VI. Discussion

Our experiments show that heterogeneous relational modeling with an RGCN yields a modest but meaningful improvement in AUPRC over both traditional ML and other GNN architectures on the diabetes readmission task. The key design choices that contributed to the RGCN's success include:

- Encoding ICD-9 diagnoses at two levels (specific code and grouped category), allowing the model to share information among clinically related conditions.
- Representing medications, physician specialty, and administrative codes as separate node types, rather than collapsing everything into flat features.
- Using a single RGCN layer with basis decomposition, which captures one-hop relational information while remaining relatively simple and stable to train.

Nonetheless, the absolute performance remains limited: even the best model achieves precision of only about 21% at recall 25%. This suggests that many factors driving readmission, such as socioeconomic status, adherence, and post-discharge support, are not captured in the dataset. Future work could extend the graph to incorporate additional data sources (e.g., outpatient visits, social determinants) and temporal patterns (e.g., linking multiple encounters for the same patient).

VII. Conclusion and Team Contributions

We presented a full pipeline for predicting 30-day readmission in diabetic patients using a heterogeneous RGCN, from preprocessing and graph construction to training, hyperparameter tuning, and evaluation. Only the RGCN among the GNN models considered achieved a test AUPRC greater than 0.15, outperforming both GraphSAGE, HGT, and strong tabular baselines.

Team Contributions and Code Availability

This project was conducted by a single author.
All code and data are available at:

https://github.com/carloea2/project273a

Appendix A
Additional Exploratory Figures

For completeness, we include two additional exploratory plots that help characterize the dataset and model behaviour:

- Figure 4: Number of medications by 30-day readmission outcome.
- Figure 5: Correlation structure among numeric covariates.

These figures support the modeling choices discussed in the main text.

References

[1] B. Strack, J. Deshpande, C. R. Faris, et al., "Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000+ patient records," *BioMed Research International*, 2014.
[2] O. G. Emi-Johnson, K. J. Nkrumah, "Predicting 30-day hospital readmission in patients with diabetes using machine learning on EHR data," *Cureus*, 2025.
[3] M. S. Magboo and A. D. Coronel, "30-day hospital readmission prediction model for diabetic patients (30–70 age group)," in *Proc. Academics World Int. Conf.*, 2019.
[4] M. Schlichtkrull, T. N. Kipf, et al., "Modeling relational data with graph convolutional networks," in *Proc. ESWC*, 2018.
[5] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proc. WWW*, 2020.
[6] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.
[7] J. Clore, K. Cios, J. DeShazo, and B. Strack, "Diabetes 130-US Hospitals for Years 1999–2008," UCI Machine Learning Repository, 2014. [Online]. Available: https://doi.org/10.24432/C5230J.